Modelo de domínio

Classes Java de atendimento, equipe, internação e cirurgia

ConsultaRotina

```
+ validarAtendimento(): boolean
```

ConsultaEspecializada

```
- especialidade: String
+ validarAtendimento(): boolean
```

AtendimentoEmergencia

```
- prioridade: Prioridade
+ validarAtendimento(): boolean
```

MedicoClinico

```
- encaminhamentos: List<Encaminhamento> = []
+ registrarEncaminhamento(encaminhamento: Encaminhamento): void
```

MedicoEspecialista

```
- especialidade: String
```

Cirurgiao

```
- areaCirurgica: String
- procedimentos: List<ProcedimentoCirurgico> = []
+ registrarProcedimento(procedimento: ProcedimentoCirurgico): void
```

«abstract»

Consulta

```
# id: Long
# data: LocalDate
# horario: LocalTime
# paciente: Paciente
# profissional: Profissional
# motivo: String
# observacoes: String
# status: StatusConsulta = AGENDADA
+ validarAtendimento(): boolean
```

Paciente

```
- id: Long
- nome: String
- cpf: String
- nascimento: LocalDate
- telefone: String
- email: String
```

«abstract»

Profissional

```
# id: Long
# nome: String
# registroProfissional: String
# uf: String
# telefone: String
# email: String
```

Quarto

```
- id: Long
- numero: String
- tipo: TipoQuarto
- capacidade: int = 1
- disponivel: boolean = true
```

Internacao

```
- id: Long
- paciente: Paciente
- quarto: Quarto
- leito: String
- medicoResponsavel: Profissional
- dataEntrada: LocalDate
- dataSaida: LocalDate
- motivo: String
- status: StatusInternacao = ATIVA
- observacoes: String
+ estaAtiva(): boolean
```

Encaminhamento

```
- id: Long
- paciente: Paciente
- medicoSolicitante: MedicoClinico
- profissionalDestino: Profissional
- especialidade: String
- motivo: String
- data: LocalDate = LocalDate.now()
```

ProcedimentoCirurgico

```
- id: Long
- descricao: String
- tipoCirurgia: String
- observacoesPreOperatorias: String
- observacoesPosOperatorias: String
- paciente: Paciente
- cirurgiao: Cirurgiao
- dataPrevista: LocalDate
- horarioPrevisto: LocalTime
- dataReal: LocalDate
- horarioReal: LocalTime
- status: StatusCirurgia = AGENDADO
+ iniciar(): boolean
+ cancelar(): boolean
+ concluir(): boolean
```

ENUMERAÇÕES

Referenciadas pelos tipos dos atributos nas classes acima.

«enum»

StatusConsulta

```
AGENDADA
CONFIRMADA
EM_ATENDIMENTO
CONCLUIDA
CANCELADA
NAO_COMPARECEU
```

«enum»

Prioridade

```
BAIXA
MODERADA
ALTA
CRITICA
```

«enum»

TipoQuarto

```
ENFERMARIA
APARTAMENTO
UTI
```

«enum»

StatusInternacao

```
ATIVA
ALTA_PREVISTA
ENCERRADA
```

«enum»

StatusCirurgia

```
AGENDADO
EM_ANDAMENTO
CONCLUIDO
CANCELADO
```

- privado # protegido + público Triângulo aberto indica herança → associação navegável

Acessores e construtores omitidos.

API e serviços

Os controllers delegam aos serviços. As operações de CRUD e o armazenamento em memória ficam em CrudService<T>.

«RestController» /pacientes

PacienteController

```
- service: PacienteService {readOnly}
```

```
+ listar(): Collection<Paciente>
+ buscar(id: Long): Paciente
+ cadastrar(item: Paciente): Paciente
+ atualizar(id: Long, item: Paciente):
  Paciente
+ excluir(id: Long): void
```

«RestController» /profissionais

ProfissionalController

```
- service: ProfissionalService
  {readOnly}
```

```
+ listar(): Collection<Profissional>
+ buscar(id: Long): Profissional
+ cadastrar(item: Profissional):
  Profissional
+ atualizar(id: Long, item:
  Profissional): Profissional
+ excluir(id: Long): void
```

«RestController» /consultas

ConsultaController

```
- service: ConsultaService {readOnly}
```

```
+ listar(): Collection<Consulta>
+ buscar(id: Long): Consulta
+ cadastrar(item: Consulta): Consulta
+ atualizar(id: Long, item: Consulta):
  Consulta
+ excluir(id: Long): void
```

«RestController» /internacoes

InternacaoController

```
- service: InternacaoService {readOnly}
```

```
+ listar(): Collection<Internacao>
+ buscar(id: Long): Internacao
+ cadastrar(item: Internacao):
  Internacao
+ atualizar(id: Long, item: Internacao):
  Internacao
+ excluir(id: Long): void
```

«RestController» /quartos

QuartoController

```
- service: QuartoService {readOnly}
```

```
+ listar(): Collection<Quarto>
+ buscar(id: Long): Quarto
+ cadastrar(item: Quarto): Quarto
+ atualizar(id: Long, item: Quarto):
  Quarto
+ excluir(id: Long): void
```

«Service» T = Paciente

PacienteService

```
# getId(item: Paciente): Long
# setId(item: Paciente, id: Long): void
```

«Service» T = Profissional

ProfissionalService

```
# getId(item: Profissional): Long
# setId(item: Profissional, id: Long):
  void
```

«Service» T = Consulta

ConsultaService

```
# getId(item: Consulta): Long
# setId(item: Consulta, id: Long): void
# validar(item: Consulta): void
```

«Service» T = Internacao

InternacaoService

```
# getId(item: Internacao): Long
# setId(item: Internacao, id: Long):
  void
# validar(item: Internacao): void
```

«Service» T = Quarto

QuartoService

```
# getId(item: Quarto): Long
# setId(item: Quarto, id: Long): void
# validar(item: Quarto): void
```

«abstract»

CrudService<T>

```
# dados: Map<Long, T> = new LinkedHashMap<>() {readOnly}
- sequencia: AtomicLong = new AtomicLong(1) {readOnly}
```

```
# getId(item: T): Long {abstract}
# setId(item: T, id: Long): void {abstract}
# validar(item: T): void
+ listar(): Collection<T>
+ buscar(id: Long): T
+ cadastrar(item: T): T
+ atualizar(id: Long, item: T): T
+ excluir(id: Long): void
```

«SpringBootApplication»

HospitalApplication

```
+ main(args: String[]): void {static}
```

- privado # protegido + público Triângulo aberto indica herança → associação navegável

T indica o tipo de domínio gerenciado pelo serviço.